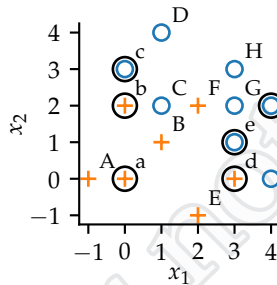


Chapter 9

Answer 9.1

Let's label the training points (in capital letters) and the validation points (in lowercase letters):



With $k = 1$, we have nearest neighbors of
validation pt closest training pt correct?

a(+)	A(+)	yes
b(+)	C(o)	no
c(o)	C(o)/D(o)	yes
d(+)	I(o)	no
e(o)	G(o)	yes
f(o)	G(o)	yes

Thus, we have a validation error rate of 2/6.

With $k = 3$, we have nearest neighbors of
validation pt closest training pts correct?

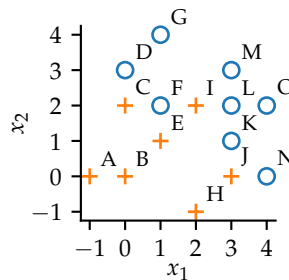
a(+)	A(+), B(+), C(o)/E(+)	yes
b(+)	C(o), B(+), F(+)	yes
c(o)	C(o), D(o), B(+)/F(+)	yes
d(+)	I(o), E(+), G(o)	no
e(o)	G(o), F(+), I(o)	yes
f(o)	G(o), H(o), F(+)/I(o)	yes

Thus, we have a validation error rate of 1/6.

We select $k=3$ because it has the lower validation error rate.

Answer 9.2

Let's label the training points (in capital letters):



With $k = 1$, we have nearest neighbors of

validation pt	closest training pt	correct?
A(+)	B(+)	yes
B(+)	A(+)	yes
C(+)	D(o)/F(o)	no
D(o)	C(+)	no
E(+)	F(o)	no
F(o)	C(+)/E(+)/I(+)	no
G(o)	D(o)	yes
H(+)	J(+)	yes
I(+)	F(o)/L(o)	no
J(+)	K(o)/N(o)	no
K(o)	J(+)/L(o)	maybe
L(o)	I(+)/K(o)/M(o)/O(o)	maybe
M(o)	L(o)	yes
N(o)	J(+)	no
O(o)	L(o)	yes

Thus, we have a validation error rate of between 7/15 and 9/15, depending on how ties are broken.

With $k = 3$, we have nearest neighbors of

validation pt	closest training pts	correct?
A(+)	B(+), C(+), E(+)	yes
B(+)	A(+), E(+), C(+)	yes
C(+)	D(o), F(o), E(+)	no
D(o)	C(+), F(o), G(o)	yes
E(+)	F(o), B(+)/C(+)/I(+)	yes
F(o)	C(+), E(+), I(+)	no
G(o)	D(o), F(o), C(+)/I(+)/M(o)	yes
H(+)	J(+), B(+)/E(+)/K(o)/N(o)	maybe
I(+)	F(o), L(o), E(+)/K(o)/M(o)	no
J(+)	K(o), N(o), H(+)	no
K(o)	J(+), L(o), I(+)/N(o)/O(o)	maybe
L(o)	I(+)/K(o)/M(o)/O(o)	yes
M(o)	L(o), I(+), O(o)	yes
N(o)	J(+), K(o), O(o)	yes
O(o)	L(o), K(o), M(o)	yes

Thus, we have a validation error rate of between 4/15 and 6/15, depending on how ties are broken.

We select $k=3$ because it has the lower validation error rate.

Answer 9.3

Note that 1 is so much less than 10 for this dataset, that the axis with a scaling of 1 is only used to break ties. For instance, for scaling I, it matters most which points are closest in x_1 . For this dataset, many points have the same value for x_1 so we pick the closest point along x_2 from those with the same value for x_1 . That is, for scaling I this amounts to picking the closest point in the same “column.”

Using the same labeling of the training points as above,

With scaling I (10.0, 1.0), we have nearest neighbors of

validation pt	closest training pt	correct?
A(+)	B(+)	yes
B(+)	C(+)	yes
C(+)	D(o)	no
D(o)	C(+)	no
E(+)	F(o)	no
F(o)	E(+)	no
G(o)	F(o)	yes
H(+)	I(+)	yes
I(+)	H(+)	yes
J(+)	K(o)	no
K(o)	J(+)/L(o)	maybe
L(o)	K(o)/M(o)	yes
M(o)	L(o)	yes
N(o)	O(o)	yes
O(o)	N(o)	yes

Thus, we have a validation error rate of between 5/15 and 6/15, depending on how ties are broken.

With scaling II (1.0, 10.0), we have nearest neighbors of

validation pt	closest training pt	correct?
A(+)	B(+)	yes
B(+)	A(+)	yes
C(+)	F(o)	no
D(o)	M(o)	yes
E(+)	K(o)	no
F(o)	C(+)/I(+)	no
G(o)	D(o)	yes
H(+)	J(+)	yes
I(+)	F(o)/L(o)	no
J(+)	N(o)	no
K(o)	E(+)	no
L(o)	I(+)/O(o)	maybe
M(o)	D(o)	yes
N(o)	J(+)	no
O(o)	L(o)	yes

Thus, we have a validation error rate of between 7/15 and 8/15, depending on how ties are broken.

We select scaling I because it has the lower validation error rate.

Answer 9.4

- a. Each point must be compared to all of the other points. This results in $m(m-1)$ distance calculations (one for each comparison).

- b. Each of the 10 folds has $0.1m$ points. Each of these points must be compared to the non-validation points, of which there are $0.9m$. Therefore, each fold requires $0.1m \times 0.9m = 0.09m^2$ distance calculations. Across all ten folds, this totals $10 \times 0.09m^2 = 0.9m^2$ distance calculations.
- c. Leave-one-out cross-validation is more expensive if $m(m-1) > 0.9m^2$:

$$m^2 - m > 0.9m^2$$

$$0.1m^2 > m$$

$$m > 10$$

or when there are more than 10 points.

Generally this means that leave-one-out cross-validation is always more expensive (because it is only less expensive when there are not enough data points to do 10-fold cross-validation). Leave-one-out cross-validation will always be more expensive because if the number of folds (k) is less than m , each point is compared to fewer than every other point and therefore fewer than in leave-one-out cross validation.

Answer 9.5

- a. b.

Python solution:

```
import numpy as np

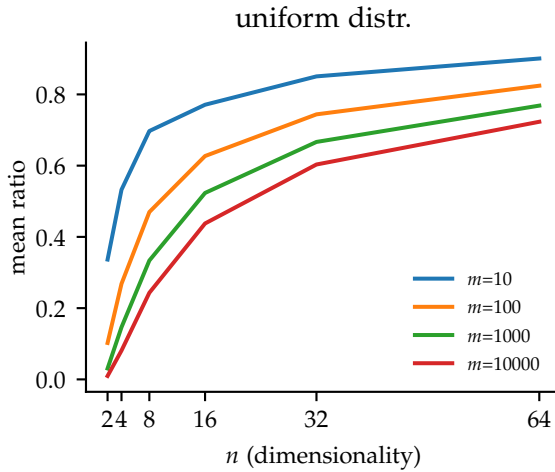
def distratio(X):
    # below faster than
    # D = ((X[:,None,:]-X[None,:,:])**2).sum(axis=2)
    d = (X**2).sum(axis=1)
    D = d[:,None]+d[None,:] - 2*X@X.T
    D[D<0] = 0 # necessary to account for fp rounding
    D = np.sqrt(D)
    avgdist = D.sum()/D.shape[0]/(D.shape[1]-1)
    np.fill_diagonal(D,np.inf)
    mndist = D.min(axis=1).mean()
    return mndist/avgdist
```

- c. `import matplotlib.pyplot as plt`

```
ms = [10,100,1000,10000]
ns = [2,4,8,16,32,64]
sampler = np.random.default_rng().uniform
nrep = 5
for m in ms:
    ratios = [np.array([distratio(sampler(size=(m,n)))
                        for _ in range(nrep)]).mean() for n in ns]
    plt.plot(ns,ratios,label=f'm={m}')

plt.legend()
plt.xticks(ns)
plt.xlabel('n (dimensionality)')
```

```
plt.ylabel('mean ratio')
plt.title(f'uniform distr.')
plt.show()
```



- d. As the number of features grows, the nearest neighbor is not that much closer than a random point. The number of features does not have to be that large for this to be true. Adding more data helps make a larger separation, but orders of magnitude more data is needed for each few extra dimensions added.

Answer 9.6

- a. If the Bayes-optimal error rate is b , then with probability b the nearest point is of the “wrong class” (the one that the Bayes-optimal classifier would not pick) and NN would be wrong $1 - b$ of the time. Conversely, with probability $1 - b$, the nearest point is of the “right class” (the one that the Bayes-optimal classifier would pick) and NN would be wrong b of the time. In total, this means that the asymptotic error rate for NN is $b(1 - b) + (1 - b)b$ which equals $2b(1 - b)$.
- b. If there are C classes, we know the most common class occurs with probability $1 - b$. The other classes occur with probabilities $p_1, \dots, p_{C-1}$ which must sum to b . Taken together, the error rate is therefore

$$\text{err}_{\text{NN}} = (1 - b)b + \sum_{j=1}^{C-1} p_j(1 - p_j)$$

To maximize, we form the Lagrangian (to enforce the constraint $\sum_{j=1}^{C-1} p_j = b$):

$$J = (1 - b)b + \sum_{j=1}^{C-1} p_j(1 - p_j) + \lambda \left(b - \sum_{j=1}^{C-1} p_j \right)$$

which has derivatives

$$\frac{\partial J}{\partial p_j} = 1 - 2p_j - \lambda$$

and therefore at the maximum

$$\begin{aligned} 0 &= \frac{\partial J}{\partial c_j} \\ &= 1 - 2p_j - \lambda \\ p_j &= \frac{1 - \lambda}{2} . \end{aligned}$$

Thus, in the worst case, p_j is the same for all j , which means that $p_j = \frac{b}{C-1}$. This gives the final formula of

$$\begin{aligned} \text{err}_{\text{NN}} &= (1-b)b + \sum_{j=1}^{C-1} \frac{b}{C-1} \left(1 - \frac{b}{C-1}\right) \\ &= (1-b)b + b \left(1 - \frac{b}{C-1}\right) \\ &= b \left(2 - b \frac{C}{C-1}\right) \end{aligned}$$

c. b and $\frac{C}{C-1}$ are greater than 0. Therefore $b \left(2 - b \frac{C}{C-1}\right) \leq 2b$.

d.

$$\begin{aligned} \text{err}_{\text{NN}} - b &= b - b^2 \frac{C}{C-1} \\ &\leq b - b^2 \\ &\leq \frac{1}{2} - \frac{1}{4} \\ &= \frac{1}{4} \end{aligned}$$

where the last inequality is true because $0 \leq b \leq \frac{1}{2}$.